Lecture 03 | July, 2026

Search Parameter Tuning

Instructor: Sithvothy Kiv

Slides Credit: Josh Hug

Search Parameter Tuning

- **Search Parameter Tuning**
- Trying different boosts
- Tuning Workflow
- Top-K tradeoffs

Search Parameter Tuning

In the previous lesson, we defined Hit Rate, MRR, and the evaluate function. Now we can use them to tune search parameters.

Instead of guessing which settings are better, we measure them on the ground truth dataset.

So far we've boosted question to 3.0. The idea was that a query should match the FAQ question. That kind of match should count for more than matching the answer text. It sounds reasonable.

But it's a guess, and now we can check it against data instead of trusting it.

Search Parameter Tuning

This is the main benefit of offline evaluation. We change one parameter, run the same questions again, and see whether the metric moves.

The dataset stays fixed, so the comparison is fair.

Trying different boosts

- Search Parameter Tuning
- **Trying different boosts**
- Tuning Workflow
- Top-K tradeoffs

Trying different boosts

Start with a search function where the question boost is configurable:

```
def search_boost(query, question_boost):
    boost_dict = {"question": question_boost, "section": 0.5}

    return index.search(
        query,
        num_results=5,
        boost_dict=boost_dict,
    )
```

Trying different boosts

Evaluate several boost values:

```
for boost in [0.5, 1.0, 3.0, 5.0, 10.0]:
    result = evaluate(
        ground_truth,
        lambda query, boost=boost: search_boost(query, boost)
    )
    print(f"boost={boost}: {result}")
```

Trying different boosts

Evaluate several boost values:

```
boost=0.5: {'hit_rate': 0.9113924050632911, 'mrr': 0.800548523206751}
boost=1.0: {'hit_rate': 0.9240506329113924, 'mrr': 0.8139240506329113}
boost=3.0: {'hit_rate': 0.8987341772151899, 'mrr': 0.7693248945147676}
boost=5.0: {'hit_rate': 0.8708860759493671, 'mrr': 0.7401265822784809}
boost=10.0: {'hit_rate': 0.8582278481012658, 'mrr': 0.7122362869198313}
```

Trying different boosts

Increasing the question boost makes the metrics worse, not better. The best value here is 1.0, no boost at all.

That's already the opposite of what the intuition predicted. But this is only one parameter.

We can also tune answer and section together with question.

Trying different boosts

Now do a small grid search:

```
results = []

for question_boost in [1.0, 2.0, 5.0]:
    for answer_boost in [1.0, 2.0, 4.0, 10.0]:
        for section_boost in [0.1, 0.2, 0.5]:
            print(
                f"Evaluating question_boost={question_boost},"
                f" answer_boost={answer_boost},"
                f" section_boost={section_boost}..."
            )
            result = evaluate(
                ground_truth,
                lambda query, question_boost=question_boost, answer_boost=answer_boost, section_boost=section_boost: search_boosts(
                    query,
                    question_boost,
                    answer_boost,
                    section_boost
                )
            )

            results.append({
                "question": question_boost,
                "answer": answer_boost,
                "section": section_boost,
                "hit_rate": result["hit_rate"],
                "mrr": result["mrr"],
            })
```

Trying different boosts

Sort by MRR:

```
df_results = pd.DataFrame(results)
df_results.sort_values("mrr", ascending=False).head(10)
```

Trying different boosts

For the same data, the best rows are:

question	answer	section	hit_rate	mrr
1.0	2.0	0.1	0.975	**0.885**
2.0	4.0	0.2	0.975	**0.885**
5.0	10.0	0.5	0.975	**0.885**
5.0	10.0	0.2	0.975	0.884
5.0	10.0	0.1	0.975	0.884
2.0	4.0	0.1	0.975	0.884
2.0	4.0	0.5	0.977	0.884
1.0	2.0	0.2	0.977	0.884
1.0	2.0	0.5	0.965	0.862
1.0	4.0	0.1	0.970	0.862

Trying different boosts

The best combination weights answer twice as heavily as question, with almost no weight on section. So the data says the opposite of where we started.

The answer text matters more for retrieval than the question text.

The intuition was wrong, and we'd never have known without measuring it. This is exactly why we evaluate instead of guess.

Trying different boosts

The first three rows have the same relative weights:

```
question : answer : section = 1 : 2 : 0.1
```

So we can use the smaller and easier-to-read values: question=1.0, answer=2.0, and section=0.1.

This gives the same relative weights as question=5.0, answer=10.0, and section=0.5, but the numbers are not unnecessarily large.

Trying different boosts

Define the search function with these boosts:

```
def text_search(query):
    boost_dict = {
        "question": 1.0,
        "answer": 2.0,
        "section": 0.1,
    }

    return index.search(
        query,
        num_results=5,
        boost_dict=boost_dict,
    )
```

Trying different boosts

Usually we care about both metrics.

- Hit Rate tells us whether the correct document appears at all.
- MRR tells us whether it appears near the top.

A document near the top is more likely to be used by the RAG prompt.

Tuning Workflow

- Search Parameter Tuning
- Trying different boosts
- **Tuning Workflow**
- Top-K tradeoffs

Tuning Workflow

Search parameters can look arbitrary. This includes field boosts, number of results, filters, and other settings. Evaluation gives us a way to compare settings with evidence.

Grid search is fine when there are only a few settings. For a larger parameter space, use a smarter search strategy.

You can sample random combinations, use Bayesian optimization, or keep a validation split so you don't overfit the evaluation set.

Tuning Workflow

For text search on our dataset, grid search takes about one second per combination. That makes it practical to try many options. When each evaluation takes minutes instead of seconds, grid search becomes too expensive.

In those cases, use Bayesian optimization with a library like hyperopt. It explores the parameter space more efficiently by focusing on combinations that are likely to improve the metric.

Top-K tradeoffs

- Search Parameter Tuning
- Trying different boosts
- Tuning Workflow
- **Top-K tradeoffs**

Top-K tradeoffs

We return 5 results from search. Increasing top-K to 10 would improve hit rate because there are more chances to find the correct document. But more results means more context sent to the LLM.

That costs more and makes it harder for the model to identify what is relevant. Five results is a reasonable default for short FAQ-style documents.

Next, we'll move from retrieval quality to answer quality and evaluate the full RAG pipeline.